USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

POPS: From History to Mitigation of DNS Cache Poisoning Attacks

Yehuda Afek, *Tel Aviv University;* Harel Berger, *Ariel University;*
Anat Bremler-Barr, *Tel Aviv University*

https://www.usenix.org/conference/usenixsecurity25/presentation/afek

This paper is included in the Proceedings of the
34th USENIX Security Symposium.

August 13–15, 2025 • Seattle, WA, USA

978-1-939133-52-6

Open access to the Proceedings of the
34th USENIX Security Symposium is sponsored by USENIX.

POPS: From History to Mitigation of DNS Cache Poisoning Attacks

Yehuda Afek
Tel Aviv University
afek@tauex.tau.ac.il

Harel Berger
Ariel University
harelb@ariel.ac.il *

Anat Bremler-Barr
Tel Aviv University
anatbr@tauex.tau.ac.il

Abstract

We present a novel yet simple and comprehensive DNS cache POisoning Prevention System (POPS), designed to integrate as a module in Intrusion Prevention Systems (IPS). POPS addresses statistical DNS poisoning attacks—documented from 2002 to the present—and offers robust protection against similar future threats. It comprises a detection module, which employs three simple rules, and a mitigation module that leverages the TC flag in the DNS header to enhance security. Once activated, the mitigation module has zero false positives or negatives, correcting any such errors on the side of the detection module. Thus, the detection module is allowed to err on the false positive side while minimizing false negatives.

We first analyze POPS against historical DNS services and attacks, showing that it would have mitigated all network-based statistical poisoning attacks. We then simulate POPS on traffic benchmarks (PCAPs) incorporating current potential network-based statistical poisoning attacks, and benign PCAPs; the simulated attacks still succeed with a probability of 0.0076%. This occurs because five malicious packets go through before POPS detects the attack and activates the mitigation module. In addition, POPS completes its task using only 20%–50% of the time required by other tools (e.g., Suricata or Snort), and after examining just 5%–10% as many packets. It successfully detects DNS cache poisoning attacks—including fragmentation-based variants—that Suricata and Snort consistently miss, highlighting POPS's superiority.

1 Introduction

One of the cornerstones of our digital world is the Domain Name System (DNS) [1, 2], which translates human-readable domain names into IP addresses. As the Internet's reliance on DNS grows, so does the complexity of this essential service. Continual extensions, refinements [3, 4], and security and privacy enhancements [5, 6] add to this ever-increasing complexity, demanding a structured, comprehensive approach to effectively address emerging challenges.

In a DNS cache poisoning attack, an attacker injects false information into a DNS resolver's cache. This manipulation causes the resolver to return the malicious IP address for a target domain name, redirecting users to malicious websites without their knowledge. The attack can serve as a gateway to phishing [7], credential theft [8], malware infections [9], and other online threats. DNS cache poisoning remains one of the most persistent DNS-based attacks [8–11], despite multiple attempts at mitigation [12, 13].

There are two primary types of DNS poisoning attacks. The first and more popular method involves delivering spoofed authoritative responses to a resolver by faking the IP address of an authoritative name server, and guessing parameters such as the DNS transaction ID and source port to match a corresponding query issued by the resolver. The second uses advanced methods—like BGP hijacking [14] or MITM attacks [15]—to bypass these guesswork steps, but is more complex to execute and therefore less common. Our study focuses on the former, more prevalent type of attack.

Proposed in 1997, the DNS Security Extensions (DNSSEC) protocol was designed to eliminate DNS cache poisoning attacks through cryptographic authentication. However, with limited global adoption (<30%)[1] and dependence on authoritative server implementation, DNSSEC has yet to achieve universal protection. Partial mitigations, such as increased randomness [13] and machine learning [16], address earlier attack vectors but prove less effective against more advanced threats [17, 18].

In the absence of a universal solution, DNS vulnerabilities are addressed reactively: when a new DNS cache poisoning attack is discovered, it usually receives a Common Vulnerabilities and Exposures (CVE) identifier, prompting vendors to release updates. This reactive cycle requires ongoing vigilance and frequent patches to maintain security.

*Most work was done during the author's time at Georgetown University.

[1] https://www.myrasecurity.com/en/knowledge-hub/dnssec/

In this paper we address the challenges posed by DNS cache poisoning attacks, with a novel mitigation system called POisoning Prevention System (POPS). The system consists of two components: a detection module (Section 4.1) and a mitigation module (Section 4.2). The detection module applies three simple rules, derived from a comprehensive analysis of DNS cache poisoning attacks throughout history, effectively capturing the unique attributes of statistical DNS cache poisoning. The mitigation module uses the TC flag in the DNS header to force suspicious request/response transactions to switch from UDP to TCP—a more secure, connection-oriented protocol whose stateful handshake and sequence number management prevent spoofing (in addition to session hijacking and DDoS attacks). While the TC flag has been previously employed to mitigate spoofed DNS-based DDoS attacks [19, 20] and to detect compromised clients [21], its potential role in mitigating DNS cache poisoning attacks has, to the best of our knowledge, not yet been explored. An earlier deployment of a prototype of POPS, could have blocked with high likelihood all the attack vectors explored in this study, including CVEs that were discovered from 2012 until today (Employing the rules that are based only on the 2002 attack, could have prevented 60% of subsequent DNS cache poisoning attacks up to the present time).

Two important practical properties that follow from our two-stage system are; (1) ensuring that suspected DNS cache poisoning packets are verified through a TCP session in the mitigation module, effectively eliminating possible false positives from the first stage; and (2) allowing the detection module to err in the false-positive direction to minimize false negatives, which decreases the attacker's success rate to less than 1%. Our system mitigates attacks and vulnerabilities described in academic literature and in DNS cache poisoning-related CVEs (e.g., [22, 23]).

The approach of building a system to prevent new attack vectors is also evident in the industry. For example, Akamai reported that their retrospective analysis enabled their defense systems to block future attack vectors, although they did not provide specific details about the solution [24].

In summary, the contributions of this work are:

- **Quick and accurate detection with Minimal Overhead.** Detecting poisoning attacks in under 1 second, after observing the first five packets.

- **Mitigation with Zero False Positives.** POPS leverages the TC bit to switch suspected responses from UDP to TCP, blocking only responses that indeed masquerade the legitimate authoritative server.

The remainder of this paper is organized as follows. Background for this work is provided in Section 2, including the process of DNS resolution, the attacker's threat model, and the types of attacks examined in our work. Section 3 presents the historical DNS cache poisoning attacks and mitigation methods Our methodology is given in Section 4, including its analysis, and implementation. The metrics used to assess our system are also presented in this section. The experiments are described in Section 5. The results are presented in Section 6. A list of the CVEs that our system would have rendered obsolete is provided in Section 7. A comparison of our system to Suricata and Snort is presented in Section 8. A discussion on our system can be found in Section 9. We conclude our work and suggest future work in Section 10. Ethical considerations are elaborated in Section 11, and compliance with open science policy is provided in Section 12.

2 Background

2.1 DNS resolution and poisoning attacks

When a DNS resolver receives a client query for domain D, it first checks whether D is present in its cache. If not, it initiates a resolution process by querying a sequence of name servers (NS) in the hierarchical DNS system, starting from the root and continuing through the appropriate top-level and intermediate domain servers, until it reaches the authoritative name server responsible for domain D. The authoritative NS responds with the IP address associated with D, back to the querying resolver. Upon receiving the response, the resolver validates that the response UDP packet has the correct source port, transaction ID (TXID), domain name (D), and that the source IP address is the IP address of the authoritative NS. If validated, the resource record (RR) in the response packet is stored in the cache, with the corresponding time-to-live (TTL) value, and the corresponding IP address (A or AAAA type part of the record) is delivered to the client.

In a cache poisoning attack an attacker inserts into the resolver cache an incorrect mapping of a domain name to an IP address of a malicious server (e.g., mimicking central bank server). In the main poisoning attack variant it is done by tricking the resolver to accept a fraudulent response as if it came from the legitimate authoritative NS with the intended malicious mapping. Mounting a poisoning attack for domain D involves variations on the following steps (see Figure 1):

1. Trick the resolver to query the authoritative NS of D, expecting a response with the IP address of D. This is achieved by a client controlled by the attacker, querying the resolver for the IP address of D.

2. If D is not in it's cache the resolver goes down the DNS hierarchy to find the authoritative server for D.[2] Alternatively, the attacker can force the resolver to resolve D, by querying a fake subdomain of D which is not in the cache [13].

[2]The attacker can remove the record for D from the resolver cache, if such exists, by e.g., the CachFlush technique of [25].

3. The resolver queries D's authoritative NS for D's address.

4. While the resolver is expecting a response from the authoritative NS, the attacker provides it with a forged response with the source IP of the authoritative NS. For the forged response to be accepted by the resolver as an authentic response, it must have the correct port-number, TXID, and domain D, in addition to the correct IP address of the authoritative NS. The port number and TXID, which are randomly generated by the resolver, are not known to the attacker who has to guess them.

5. If the attacker's forged (poison) response is accepted by the resolver, it is stored in the cache with the fake (poison) mapping of D (directly, or via a glue record [13]).

We distinguish between four major types of statistical poisoning attacks; S, S_{Frag}, B_{Frag}, S_{OoB}, some of the types may be variants of the steps above:

In type S (Statistical), step 4 involves the attacker sending numerous combinations of the source port and TXID numbers, hoping that one will match the correct values. If neither parameter can be obtained, the probability of success is $\frac{1}{2^{32}}$. If the attacker can correctly determine one of the parameters, the probability rises to $\frac{1}{2^{16}}$. The attack strategy relies on brute-force injection of forged responses, each attempting to guess the correct port-TXID pair.[3]

In type S_{Frag}, fragmentation is used as follows [27]: The attacker starts with step 4, in which it sends a forged 2*nd* fragment containing the poison mapping it wants to be cached. Then, it follows step 2 to trick the resolver to query authoritative NS with a query whose response size does not fit into one UDP packet thus requiring fragmentation. While fragmented packets do not carry the port number nor the TXID, except for the first fragment, they do carry a fragmentation ID (IPID) a 16 bit number. When the first fragment from the authoritative NS arrives at the resolver, it appends the already existing 2*nd* fragment if their IPIDs match. The resulting response is then processed by the resolver, which unknowingly caches the malicious (poison) data from the second fragment. In this variant, the attacker thus needs to send 2^{16} different IPIDs 2*nd* fragment in order to succeed.

In type B_{Frag}, the same procedure as with S_{Frag} is followed except that it is a bullseye (B) attacker, which knows the IPID number by other means and no guessing is required.

In type S_{OoB}, the attacker exploits DNS resolvers that improperly handle DNS records in a response that pertain to domains outside the authority of the name-server that was queried —a situation referred to as "out-of-bailiwick (OoB)."

In this attack, in step 4, the attacker includes in the forged response an OoB record with the mapping it wishes to insert into the cache (e.g., a mapping for bank.com while the original query was for example.net. The .com domain is out of bailiwick for .net). Due to inadequate validation, the resolver may erroneously cache the additional OoB mappings. Consequently, future queries for subdomains of bank.com may return malicious IP addresses under the attacker's control.

2.2 Threat Model

This paper considers the threat of off-path network based DNS cache poisoning attacks, targeting resolvers during the DNS resolution process. We consider attackers mounting attacks of types S, S_{Frag}, B_{Frag}, S_{OoB}, as described above in 2.1. The resolver employs randomization techniques such as using random source ports/DNS TXID[4] to prevent an attacker from guessing these parameters. To counteract this approach, the attacker uses the techniques described above in 2.1.

Our threat model assumes the attacker brute-force predicts the DNS TXID and source port, and also the resolver lacks additional security measures that could prevent such attacks (e.g., DNSSEC). Also, the attacker is assumed to use the UDP transport protocol, except if forced otherwise by POPS.

Attacks involving man-in-the-middle (MITM) positions, BGP hijacking (e.g., [30–32]), or direct compromise of the resolver are out of scope for this study. We do not specifically consider DNS cache poisoning attacks on clients, e.g., forwarders, (e.g., [33, 34]), which generally have less impact, as they target a single client. Moreover, attacks on the DNS client can be mitigated by using our system with slight modifications. For example, embedding a syn-cookie in POPS's TCP connections [19, 20] can prevent the acceptance of the attacker's fake responses by the client. Additionally, we exclude attacks that focus on improper processing of DNS responses [23]. These attacks exploit syntactic vulnerabilities [35] and therefore require different mitigation strategies [36], which are beyond the scope of this paper.

TCP session hijacking attacks are also out of scope for this paper. These attacks rely on correctly guessing 32-bit TCP sequence numbers, which has a brute-force success probability of only 2^{-32} [37]. On its own, this makes such attacks practically infeasible. However, the threat becomes realistic when attackers possess privileged capabilities—such as host malware, compromised middleboxes, or access to exploitable side channels [38–40]. Host malware can directly access or manipulate the TCP stack; compromised middleboxes can inspect sequence numbers in transit; and side channels can leak partial information about TCP state. These capabilities drastically reduce the entropy required to successfully hijack

[3] Historically, attackers exploited the birthday paradox to boost the probability of a match. By triggering multiple simultaneous queries to the same authoritative name server, the chance of a successful collision rose significantly, from $\frac{1}{700}$ up to $\frac{1}{400}$ [26]. This vulnerability was patched by all major DNS vendors by 2003. Moreover, under our proposed mitigation, the success rate would be roughly $\frac{1}{500}$.

[4] Another existing security measurement is randomizing the capitalization of letters in the domain name [28] using the 0x20 bit, which is validated by the resolver if employed. However, there is no sufficient evidence for 0x20 bit global utility. Also, for short domain names, its effectiveness is limited [29].

a session, but fall outside the assumed threat model of this work.

3 Related Work

3.1 DNS Cache Poisoning Attacks Evolution

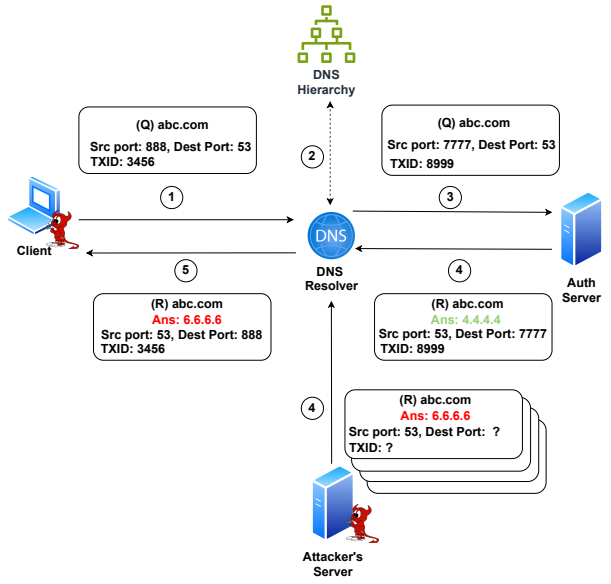

Figure 1: **Simple DNS Cache Poisoning Attack** In a typical DNS cache poisoning attack (type *S*), the following steps occur. (1) A client controlled by the attacker queries a DNS resolver for a domain such as abc.com. (2) If the resolver does not have the response in its cache, it queries the DNS hierarchy to resolve the query. (3) At the bottom of the hierarchy, the resolver queries abc.com's authoritative server. (4) In parallel, both the correct response from the authoritative server and a set of fake responses from the attacker reach the resolver. The attacker attempts to correctly guess necessary parameters—such as the transaction ID (TXID) or the source port of the resolver—to successfully poison the cache. It sends multiple responses based on the parameters it needs to guess. (5) If one of the attacker's packets has the correct parameters and arrives before the legitimate response arrives, the resolver caches this fake (poison) response and returns it to the client.

In this section, we present the DNS cache poisoning attacks from the academic literature (see Table 1). For brevity, we mention the type of the attacks in parentheses (e.g., 2008 [13] (*S*)). The first documented DNS cache poisoning attack dates back to 1993 [12], and involved creating a fake DNS response that arrives before the legitimate response. However, since this attack is performed within the resolver itself, and our threat model assumes an attacker residing outside the resolver, we did not consider its implications in our study. Subsequently, two *S* type attacks were documented by Sacramento [41] and Klein [42]. Both attacks assumed that the attacker knows the source port of the resolver because it is constant or can be determined by other means. In both the attacker uses brute-force guessing on the TXID and sends fake responses with each possible TXID to the resolver in parallel with the legitimate response. At the time the attacks were published, the security measures included only semi-randomization of the Transaction ID (TXID). Therefore, the attacker only needed to guess the 16-bit TXID correctly.

The next attack to consider is Dan Kaminsky's renowned DNS cache poisoning attack from 2008 [13] (*S*). In this attack, the attacker assumes that the resolver's source port is known and aims to poison the cache with malicious information. To achieve this, and bypass the limitation that the victim domain can be poisoned only when the TTL of the domain is expired,

Paper	Type	#Pkts	POPS
Schuba et al. [12] 1993	-	1	-
V. Sacramento [41] 2002	S	2^{16}	Rℓ1
Klein, Amit [42] 2007	S	>100	Rℓ1
Kaminsky, Dan [13] 2008	S	$200 \cdot q$	Rℓ1
Herzberg et al. [18] 2012	S	2^{16}	Rℓ1
Herzberg et al. [18] 2012	S_{Frag}	2^{16}	Rℓ2
Herzberg et al. [43] 2013	B_{Frag}	1	Rℓ2
Herzberg et al. [44] 2013	S_{Frag}	$\sim 2^{11}$	Rℓ2
Herzberg et al. [45] 2013	S, S_{Frag}	2^{16}	Rℓ1
Zheng et al. [46] 2020	B_{Frag}	1	Rℓ2
Man et al. [47] 2020	S	2^{16}	Rℓ1
Dai et al. [48] 2021	S_{Frag}	64	Rℓ1
Klein et al. [49] 2021	S	2^{16}	Rℓ1
Jeitner et al. [50] 2022	S	2^{16}	Rℓ1
Jeitner et al. [50] 2022	S	2^{16}	Rℓ1
Li et al. [17] 2023	S_{OoB}	2^{16}	Rℓ3
Heftring et al. [51] 2023	S_{Frag}	2^{16}	Rℓ2
Li et al. [22] 2024	S	2^{16}	Rℓ1

Table 1: **Significant DNS Cache Poisoning Attacks.** The 'Type' column describes the type of the attack. Two major types are described - Bullseye (B) and Statistical (S). B refers to an attack in which the targeted resolver sees only one packet the attacker generates. S denotes an attack in which the attacker sends multiple packets to the resolver. A sole S refers to an attack that involves TXID/Port guessing, S_{Frag} denotes a fragmentation attack using multiple packets, B_{Frag} refers to a fragmentation attack using a single packet, and S_{OoB} refers to "Out-of-Bailiwick", the creation of packets that violate the Bailiwick rule. The number of packets required for the attack to succeed is described in the $\#Pkts$ column. For Kaminsky's attack, the variable q denotes the number of forged subdomain queries (see explanation in the text).

the attacker generates a large volume of queries (the q value in Table 1) to non-existent subdomains of the target domain. For example, if the target domain is *abc.com*, the attacker queries the resolver for *1.abc.com*, *2.abc.com*, ..., *q.abc.com*. For each query, the attacker guesses 200 TXIDs and sends fake responses to the resolver using these guessed TXIDs. Each forged response includes glue records for the authoritative name server of the target domain (e.g., *ns.abc.com*) that maps this name server to a malicious IP address controlled by the attacker.

Following Kaminsky's attack, a new countermeasure was implemented that involved randomizing both the transaction ID (TXID) and the source port of resolver queries, requiring an attacker to guess 32 bits to successfully spoof a response. Subsequent attacks aimed to reduce this guessing requirement back to 16 bits by eliminating the need to guess one of these values. The first notable attempt to address this challenge was by Hertzberg et al. [18] (S), who focused on scenarios where the resolver is behind a Network Address Translation (NAT). They developed methods to pin or predict the NAT port, thereby reducing the need to predict both the TXID and the source port. In a subsequent study, Hertzberg [45] (S, S_{Frag}) described attacks where the name server's port is calculated using a query-generating zombie host. The attacker then sends manipulated responses, needing to guess only 16 bits. Man et al. [47] (S) proposed an attack involving a Distributed Denial of Service (DDoS) on the authoritative server, combined with a UDP/ICMP port scan to uncover the resolver's source port mechanism, followed by sending fake responses with different TXIDs. Klein et al. [49] (S) demonstrated how an attacker could determine the resolver's source port by analyzing the state of the pseudo-random number generator, then only guessing the TXID to run a cache poisoning attack. Jeitner et al. [50] (S) identified weaknesses in routers with built-in DNS resolvers, including predictable ID generation and constant source ports, which attackers could exploit by varying TXIDs or source ports. In the Tudoor attack [22] (S), the attacker first sends a benign query to the resolver. Subsequently, the attacker sends multiple DNS queries embedding port numbers within the domain name to identify the correct source port of the resolver. After determining the correct port, the attacker only needs to guess the TXID of the original benign query by sending multiple fake responses with different TXIDs.

An alternative approach to DNS cache poisoning leverages IP fragmentation to bypass the need for guessing the Transaction ID (TXID) or source ports. Hertzberg et al. [43] (B_{Frag}) pioneered this method by analyzing how resolvers generate IP identification (IPID) values; by predicting the IPID sequence, an attacker can send a single malicious second fragment that the resolver mistakenly reassembles with a legitimate first fragment, leading to cache poisoning. In subsequent work, Hertzberg et al. [44] (S_{Frag}) introduced a technique involving multiple second fragments with different IPIDs, allowing the attacker to manipulate the resolver into reassembling a legitimate first fragment with a fake second fragment by increasing the chances of IPID collision. Zheng et al. [46] (B_{Frag}) expanded on fragmentation attacks by involving both a forwarder (a DNS server that forwards DNS queries to external servers for resolution) and a resolver: in their attack, the attacker sends a malicious second fragment to the forwarder, which then reassembles it with a legitimate first fragment before forwarding it to the resolver. Dai et al. [48] (S_{Frag}) demonstrated that an attacker can induce fragmentation using ICMP packets and then send a malicious second fragment directly to the resolver; when the resolver processes a legitimate query, it reassembles the response using the attacker's fragment, resulting in cache poisoning. Heftrig et al. [51] (S_{Frag}) described an attack targeting DNSSEC responses. By first triggering fragmentation and then sending a counterfeit DNSSEC second fragment, the attacker causes the resolver to include the fake fragment in the reassembled response from the authoritative server, thus poisoning the cache even in the presence of DNSSEC.

The last approach was demonstrated by Li et al. [17] (S_{OoB}), in which an attacker induces a CDNS (a DNS server that analyzes incoming queries and conditionally forwards them to designated resolvers or forwarders for tailored responses based on its policy) to query its own domain. The attacker then sends a spoofed response, which includes an NS record that delegates a top-level domain (TLD) to the attacker's authoritative server—a clear violation of the "out of bailiwick" rule. As a result, the CDNS caches this incorrect delegation to the attacker's server.

3.2 Related Poisoning Prevention Systems

Throughout the years, several efforts have focused on hardening the DNS protocol against attacks. Probably the most famous one is DNSSEC [52], which started back in 1997. This defense allows for the cryptographic authentication of DNS records, thus preventing DNS cache poisoning attacks via counterfeit packets. DANE [53–55] utilizes the structure of DNSSEC to authenticate domains using TLS certificates. The adoption of these solutions is limited by the global implementation of DNSSEC in resolvers, which is less than 30%[5]. DNSCurve [56], introduced in 2009, attempted to encrypt DNS traffic to prevent interception of queries and responses, but it was not widely adopted. Since then, other approaches have tried to make DNS more resilient to privacy and security issues [6, 57–59]. DNS over HTTPS[6] (DoH) [59] allows DNS queries and responses to be sent over secure HTTPS traffic and has been implemented since 2018 by services from

[5] https://www.myrasecurity.com/en/knowledge-hub/dnssec/

[6] HTTPS itself may prevent users from accessing a malicious website without a valid certificate. However, as noted by Doe [60], DoH does not offer this safeguard. Furthermore, the resolver—which is the focus of this paper—is not secured when handling HTTPS traffic.

Google, Amazon, and Cloudflare. Although it allows safer transit of DNS queries, there is no transparency in how DNS queries are processed on the sender's side[7]. Although DOH can prevent attacks on the traffic between clients and resolvers, it does not prevent the poisoning of the resolver's cache. Thus, DOH does not prevent the attacks examined in our work.

Other works have abstracted the DNS protocol to detect anomalous or malicious DNS traffic. One of the earliest efforts in this direction assessed DNS cache poisoning attacks [61], focusing solely on Kaminsky's attack using the probabilistic model checker PRISM [62]. A similar approach was presented in [63]. The SPIN model checker [64] was utilized in another study with finite state machine models in [65] to detect abstracted (but non-specific) DNS cache poisoning attacks.

A DNS cache poisoning mitigation was devised by Laraba et al. [66] using Petri nets [67] and P4 programmable switches. However, it addressed only simple DNS cache poisoning, similar to Kaminsky's attack [13].

Other approaches have employed machine learning algorithms to detect DNS attacks. For instance, Anax [68] used a machine learning (ML) approach based on heuristics from 300,000 servers around the world to detect cache poisoning attacks. More details can be found in [69].

All the above-mentioned works analyzed the attacks through heuristics, patterns, and machine-learning ideas to tackle different DNS cache poisoning attacks. However, no solution tried to cover all DNS cache poisoning attacks at once. In the current work, we suggest such a solution with a set of three simple rules and a mitigation technique.

4 Poisoning Attacks Detection and Mitigation

We present here POPS's, two modules: rule-based detection (Section 4.1) and mitigation (Section 4.2). Mitigation is activated after five suspicious packets and achieves perfect accuracy—no false positives or negatives—by allowing only authentic responses through. Thus, an attack success rate is just 0.00076%. Detection may occasionally flag legitimate responses, but since these are passed to the mitigation, which blocks only spoofed responses, no valid responses are dropped — allowing near-perfect recall. At last, POPS's implementation and evaluation metrics are described in Sections 4.3 and 4.4.

4.1 Detection Module

According to Table 1, 60% of the DNS poisoning attack types examined in here are type S, involve guessing either the TXID or source ports, with 2^{16} possibilities for each. An additional 28% are type S_{Frag}/B_{Frag}, using the second fragment as an attack vector. The two exceptions, are Schuba et al. [12] which is outdated and infeasible today, and Li et al. [17], which exploits incorrect Bailiwick rule enforcement (type S_{OoB})

We thus devise three rules one of each of $\{S, S_{Frag}/B_{Frag}, S_{OoB}\}$ type, covering all relevant attacks in Table 1:

1. **Excessive Guessing of TXID/Port (Rℓ1):** We monitor the number of DNS response packets that differ from each other only in one common field—the port or the TXID, within a small window of time ($\sim 1sec$). When this number crosses a threshold (a system parameter, e.g., 5 responses) for the same DNS query, we flag these responses as a potential poisoning attack and pass this and all following responses that match the same query to the mitigation module. See Section 4.1.1 for details.

2. **Fragmentation (Rℓ2):** The first fragment of any DNS response is passed to the mitigation module. Any other fragments are discarded to prevent fragmentation-based attacks. See Section 4.1.2 for details.

3. **Out of Bailiwick (Rℓ3):** DNS responses in which the queried domain is outside the authority of the responding DNS server, are identified as potential S_{OoB} attack packets and are passed to the mitigation module. See Section 4.1.2 for details.

The last column of Table 1 summarizes the matching of each poisoning attack and a rule.

4.1.1 Rℓ1 Algorithmic Approach

In Rℓ1, we detect a high volume of packets with identical fields, except for the port or TXID. From Table 1, we see that the typical packet count for the S type is 65,535—reflecting all possible values for either the port or TXID[8]. Therefore, in Rℓ1, a method is essential to differentiate benign DNS query responses—which are usually limited to one or two packets—from malicious cases, where thousands of responses may appear for the same query. Our objective is to develop an efficient, resource-light solution with a low error rate.

To address this, we review previous heavy hitters and distinct heavy hitters works. An interesting algorithmic approach was used by Feibish et al. [70] and Nadler et al. [71] to identify anomalous behavior in networks using Heavy Hitters. Specifically, [70] used it for DNS DDoS attack detection, and [71] for DNS tunneling attack detection. Both works utilized the constant size memory of heavy hitters to address large amounts of traffic. Another useful approach was demonstrated in DNSGuard [72], an ML-based technique for mitigating DNS attacks, using Count-Min Sketch [73] (CMS).

[7] https://www.csoonline.com/article/575131/the-status-quo-for-dns-security-isn-t-working.html

[8] Historically, in attacks such as those by [13, 41, 42], the randomness of both TXID and port was minimal, allowing shorter exhaustive searches. More recent attacks (e.g., [22, 50]) circumvented guessing the TXID and source port but still required navigating through the full range of 65,535 possible TXID or port guesses.

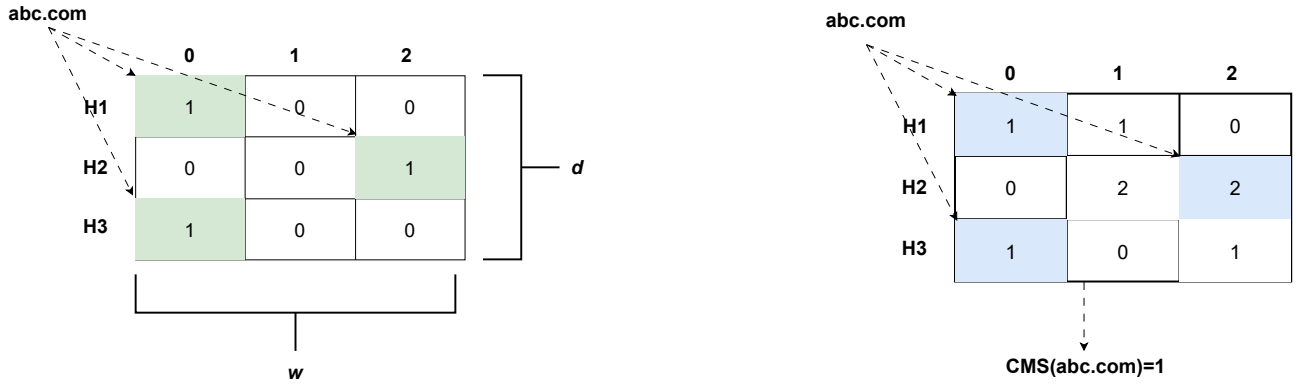

Figure 2: **CMS illustration.** CMS is defined by a number of hashed functions d, and the amount of counters per hash function w. In the subfigures, we describe each hash function as H_i. When a new element is obtained, it is hashed multiple times, and each hashed value is mapped to a cell in the respective row, increasing the counter in that cell (a). When an element is estimated, it is hashed using each one of the hash functions, gathering all values from the appropriate cells, and obtaining the minimum value of these cells (b).

We compare the three different algorithms for frequency estimation in DNS attacks: Count-Min Sketch (CMS), Fixed-size Distinct Weighted Sampling (dwsHH), and Fixed-threshold Distinct Weighted Sampling (WS). The three algorithms were compared on memory usage, error rates, and inference time for various numbers of domains (10, 100, 1000, and 10K). The comparison and analysis are relegated to Appendix A. Following the analysis, we found CMS most suitable for Rℓ1. It offers a simple and efficient way to estimate frequencies: it hashes each item into a fixed set of counters and returns the minimum value across them. This gives a tunable trade-off between memory and accuracy, ideal for high-throughput scenarios. An illustration of its structure and query process is shown in Fig. 2. Compared to the other algorithms, CMS is more memory efficient with a high volume of domains, has a low error rate, and its inference time is constant.

4.1.2 Rℓ2 & Rℓ3 Algorithmic Approach

In Rℓ2, fragments are identified as potential indicators for a DNS cache poisoning attack. Instead of storing data, we monitor the packet's offset and the MF (More Fragments) flag. When the offset equals zero and the MF flag is set (indicating the first fragment), fragmentation is detected. In Rℓ3, we follow the Bailiwick rule. Any packet that does not enforce this rule, is identified as suspicious. Here as well, we do not need to store any data for future calculation. We employ both rules without any additional resource consumption.

4.2 Mitigation Module

When a response is suspected of being poisonous, its content is being erased and the corresponding query/response conversation is moved from DNS over UDP to DNS over TCP, by setting the TC flag to true on the response that is being forwarded to the target resolver. This way the resolver acquires the response from the name server over TCP and it a guarantee that it communicates with the authentic NS due to the TCP three-way handshake, thus mitigating poisoning that does not come from the intended name server itself. The TC flag [74] was originally designed to indicate that a packet was truncated due to the packet's length (i.e., the packet is larger than the maximum transmission unit (MTU)) and thus should be discarded by the resolver, and be requested again over TCP. While the TC flag was previously used to mitigate various attacks, to the best of our knowledge, it has not yet been systematically applied to mitigate DNS cache poisoning attacks. The TC flag was used as a mechanism to mitigate spoofed DNS-based DDoS attacks [19, 20], and was also occasionally set for clients suspected of being hijacked [21].

Practically, when a suspicious response is identified, its TC flag is set to true (TC=1). Additionally, all data of the answer, authoritative (auth), and additional (add) parts of the answer are removed from the response[9]. A recent measure-

[9] In the case of Rℓ2, the fragment contains partial information. Since only the first fragment is sent to the mitigation module, we can derive the required field values from the question section within this fragment to generate the truncated response (e.g., qname, TXID, etc.). To the best of our knowledge, there is no documentation in the official RFCs [74, 75] specifying which fields must be extracted from the truncated packet. We assume that the DNS layer in the truncated packet provides the necessary information to initiate a new TCP session with the authoritative server.

ment of DNS resolvers by APNIC Labs [76] used ad-based measurement techniques to investigate the use of information from truncated packets. By examining the queries of approximately 394 million users around the world (primarily querying their default ISP's resolvers, with estimated hundreds of thousands of unique resolvers), APNIC found that 0.05% of these queries used information from truncated packets, even though such data should be disregarded. To mitigate the use of potentially compromised data, we erase any data from truncated packets before forwarding the packet to the intended resolver. Additionally, APNIC observed that 2.67% of resolvers do not resend a TCP query to the authoritative server after receiving a DNS response with the TC flag set to true. For comparison, Moura et al. [77] found a lower rate of 1.78% among resolvers in the Netherlands. An alternative mitigation option that might have addressed the issue with these non-cooperating resolvers could have involved setting the TTL value to zero, signaling that the information in the packet immediately expires. However, more than 8% of resolvers ignore this explicit expiration signal by extending the response's TTL [78], limiting the applicability of this approach. Thus, our system is not compatible with the 2.67% (or 1.78% [77]) resolvers that do not initiate a TCP session when receiving a truncated packet.

The mitigation module may send multiple packets (attacker packets) with the TC flag set to true to the resolver. Because the resolver validates both the TXID and the source port, most of these packets will not match the original query and will be discarded. The resolver will accept only the packets that match both parameters. Once a matching packet is accepted, the resolver opens a TCP connection with the authoritative NS and obtains an authentic non-poisonous resposne.

4.3 POPS algorithm

Our system's final algorithm (full pseudo code in Appendix B) follows the match-action steps [79] for processing a given set of DNS response packets P:

1. Get the CMS table, threshold τ, window size W, an initial time t (first packet), a blank map $domainMap$, and a global boolean variable $action$[10].
2. For each packet p_i in P:
   (a) If applying Rℓ1, Rℓ2, or Rℓ3 on p_i matches, and p_i is not null, set the action to "truncate". Erase the packet's answer/auth/add records and set the TC flag to true. Else, the action is "forward".
   (b) Finally, send the packet to the destination.
3. **Rℓ1**(Algorithm 2):
   (a) Add the packet's domain name to the CMS table. Thus counting the number of occurrences of this domain during the window W.
   (b) Check if the frequency of this domain in the CMS table is above the threshold τ. If so, check if this domain exists in $domainMap$ already. If so, return true. Elsewhere, if it is a new domain, save the domain in $domainMap$. Then, return true.
   (c) Periodically, based on the time value, check if we covered the whole window W. If so, empty the CMS, and set t to the time of the current packet.
   (d) Return false.
4. **Rℓ2**(Algorithm 3): If the offset of the packet equals zero and the MF flag is set to true, return true. Elsewhere, if the offset of the packet is greater than zero, the packet is nullified. Then, return false.
5. **Rℓ3:**(Algorithm 4): If the answer, authoritative, or additional records of p_i do not comply with the Bailiwick rule, return true. Elsewhere, return false.

[10] Example parameter values: threshold τ — 5, 10, 15 packets; window size W — 0.5 s, 1 s; initial time t — Monday, 12:00:00 PM; $domainMap$ = `abc.com`; $action$ = "truncate", "forward".

4.4 Metrics

We evaluated our system by measuring the attacker's success rate (ASR), a metric used to assess the effectiveness of DNS cache poisoning attempts. Given that statistical DNS cache poisoning attacks typically involve sending a high volume of packets, our goal was to determine the likelihood of an attacker's success. In this evaluation, we assume a 100% success rate if the attacker transmits all intended packets without detection. However, if only a portion of the packets are sent before detection, the success rate is scaled according to the proportion successfully transmitted packets. E.g., if only 5 out of an intended 1000 packets are sent before detection, then the success rate is 0.005.

We further evaluated our system's capability to accurately distinguish between benign and malicious packets by measuring the false positive (FP) rate, which indicates instances where benign packets are mistakenly classified as malicious.

Since the resolver opens a TCP connection in each case where the system identifies a suspicious response, it guarantees that the correct authentic resolution is obtained and is then passed on to the client. Thus, only valid responses (matching a resolver's outgoing query) will be processed, the others will be dropped. As a result, each client's query is answered reliably, even if an attacker attempts to concurrently poison the resolver. Thus, while false positives of the detection module are analyzed, they do not impact the resolution process, which together with the mitigation module has no false positive (FP).

5 Experiments

We implemented the rules and their testing framework in Golang and Python. The complete source code is included as part of the artifact accompanying this paper [80]. In the simulated environment, we collected and reconstructed a rich set of PCAP files of benign and attack packets. All simulations were executed on a desktop machine with an 11th Gen Intel® Core™ i7-1185G7 @ 3.00GHz processor and 32 GB of RAM. We carried out four experiments using this setup:

1. This experiment checks and verifies the performance of Rℓ1 implementation. We generated a single query from the attacker to the resolver, 65,535 responses for this query with a different TXID[11], and an authentic response from the authoritative server. Following [22], the attack duration is approximately 400 ms, aiming to send all guesses quickly enough to beat the authentic response. To test whether POPS can identify attacks in the presence of benign traffic, we added 1,000 randomly generated benign packets— with domains taken from the Tranco list [81]—following the normative volume of 1,000 DNS queries per second suggested by [82]. These packets simulate a more realistic network environment where benign traffic is sent to the resolver alongside the attacker's packets. Additionally, we explored different values of CMS configurations. In CMS, d is the number of hash functions, and w is the number of cells per hash function. More hash functions reduce error; more cells improve estimate accuracy. For more details, see Appendix A. We evaluated hash functions ($d = 2,3,4,5$) and cell counts ($w = 100,200,500$), omitting lower w values as they provided ineffective results.

2. This experiment mimics the 2nd fragmentation attack method, to test Rℓ2's performance.

3. This experiment generated an attack that violates the bailiwick rule, testing the performance of Rℓ3.

4. The last experiment utilized benign data from Mendeley Data [83], which comprises over 45 million DNS packets collected over a period of 1.5 days from more than 4,000 users within an academic campus. This dataset, split into multiple files by hour as described in the original study, was analyzed by focusing solely on the DNS responses received by the resolver (IP address 172.31.1.6, as documented in [83]) from servers in the DNS hierarchy. After filtering, approximately 6 million packets remained. This benign dataset served to evaluate the system's false positive (FP) rate.

[11] We test using TXID-randomized packets. To the best of our knowledge, all known DNS cache poisoning attacks require TXID prediction, either directly or after bypassing port randomization. While the specific field being guessed may vary, our detection system identifies attacks based on high volumes of response packets directed at the same domain. From the system's perspective, whether the attacker manipulates the TXID or port is interchangeable—both result in the same observable pattern. We therefore demonstrate detection using TXID variation.

In all our experiments, we set the parameters to τ=5 packets, and W=1 second, providing sufficient overhead for normal DNS traffic, typically involving 1–2 responses per query. The one-second window accommodates the most proficient attack reported by Li et al. [22] and simplifies the benign packets addition, as explained above.

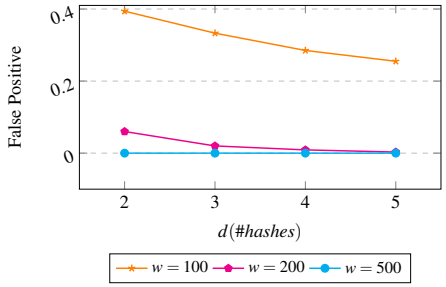

Figure 3: **False Positive Rate as a Function of d Value – Interleaved Attack & Benign Data.** Different d values for CMS, and their false positive rates for various w values. With $w = 200$ and $d = 2$, the FP rate is 6%, and with $d = 5$, the FP rate is below 1%. For $w = 500$, it is 0%.

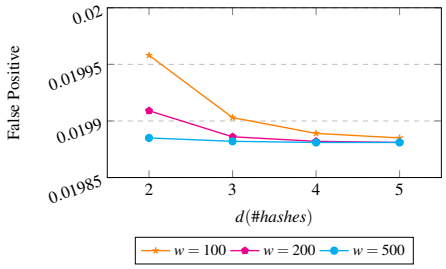

Figure 4: **False Positive Rate as a Function of the d Value – Benign Data.** Different d values for CMS, and their FP rate for w values. It is shown that the largest FP rate is less than 2%.

6 Results

6.1 Rℓ1 Experiments

Figure 3 shows the FP rates of the attack. Larger window sizes significantly reduce false positive rates, with $w = 500$ eliminating them entirely, while smaller windows like $w = 100$ yield rates exceeding 20%, especially with fewer hash functions. Across all experiments, the attack success rate (ASR) was 0.0076% and thus was not visualized. This low ASR was since after $\tau = 5$ packets. All subsequent packets with the same malicious domain within a one-second window were

identified as malicious[12]. It is worth noting that when a suspicious packet is identified by POPS, a truncated packet is sent to the resolver, prompting it to open a TCP connection to the authoritative server. Only valid packets (based on the TXID and port) will be processed. Thus, although we document an FP rate, there is no actual loss for benign requests wrongly identified as malicious; instead, the resolver obtains the result via TCP and transfers the response to the client.

6.2 Rℓ2 & Rℓ3 Experiments

POPS detects the fragmentation and out-of-bailiwick attacks, respectively. As each rule targets a single packet (Rℓ2 on the first fragment, and Rℓ3 on the out-of-bailiwick packet), the system finds this packet instantly.

6.3 Begin Data Experiment

We analyzed the FP rate of POPS on clean, benign data (i.e., without any attacks). Figure 4 illustrates the trends in these FP rates. As observed, for $w = 100$, the FP rate on benign data is approximately 2% across various values of d. As discussed in Section 6.1, this FP rate is innocuous, as the resolver successfully retrieves the result via TCP and forwards the response to the client.

In summary, the experiments demonstrated that POPS provides precise analysis of benign traffic and DNS cache poisoning attacks. With a configuration of $w = 200$ and $d = 5$, POPS achieved an ASR of just 0.0076% and a false positive rate of 2% on clean data, and maintained a low FP rate of 1% even when benign and malicious traffic was interleaved. Due to the nature of POPS, the fallback to TCP still permits successful communication and does not prevent the DNS query from ultimately being answered. To get a 0% FP, $w = 500$ and $d = 2$ can be utilized.

Following the above, the CMS can be scaled to support high-volume resolvers by increasing its allocated memory. In our experiments, a 4Kb CMS is sufficient to achieve a 0% false positive rate. Even under an extreme load of 1 billion responses per second, the CMS requires only 1,043,712 bits ($\sim$128 KB) of memory, using parameters $w = 2718$ and $d = 12$. This remains well under 1Mb, while still maintaining a compact representation with a guaranteed error bound below 0.1%. Importantly, inference time remains constant regardless of the number of tracked domains. For the complete error analysis, refer to Appendix A. These results demonstrate that POPS's use of CMS enables an efficient handling of massive-scale input while maintaining both high accuracy and minimal memory overhead.

[12]When packet arrivals span two seconds, the ASR can increase by 0.0152% due to five additional attack packets evading detection.

7 POPS as a Proactive CVE Mitigation

In this section, we present 14 CVEs related to targeted DNS software that POPS prevents from being exploited. Employing POPS reduces the necessity of fixing resolvers that use vulnerable DNS software mentioned in these CVEs. We compiled the list of CVEs from NIST [84] by searching for "DNS cache poisoning" [85] and "DNS spoofing" [86]. We excluded CVEs related to DNS clients, those that reference DNS cache poisoning or spoofing merely as a component of broader attacks without detailing the DNS-specific exploitation (e.g., [87]), and those involving DNSSEC, which we omit due to its limited deployment and reliance on authoritative servers.

We manually explored the given information and mapped which type of attack can exploit each CVE (based on the types in Section 2.1). Through this mapping, we demonstrate that POPS mitigates all reported vulnerabilities associated with the following CVEs, as it has previously addressed the corresponding types of attacks. We also described the vendor affected by each CVE. Then, we documented the rule of POPS that captures this CVE. Following is a brief description of each CVE:

1. **CVE-2023-30464 [88]** affects CoreDNS, a Golang-based DNS server library. The exploitation involves sending multiple attack packets with varying TXID and source port values, leveraging the recent Tudoor attack [22]. This exploit is thus classified as an *S* attack, similar to Tudoor.

2. **CVE-2023-28457 [89]** was also discovered through the Tudoor attack. In this case, vulnerabilities were found in Technitium DNS and Microsoft DNS, susceptible to similar exploitation as CVE-2023-30464. This vulnerability is similarly classified as an *S* attack.

3. **CVE-2023-28457 [90]** reveals a vulnerability in Technitium DNS that allows injection of NS records, even at the TLD level, due to improper validation of the bailiwick rule. This CVE is exploited through a B_{OOB} attack, as it involves a packet that violates the bailiwick rule.

4. **CVE-2021-3448 [91]** discloses a vulnerability in dnsmasq, enabling an attacker on the network to exploit a fixed port used in query forwarding, facilitating DNS cache poisoning by guessing random TXIDs. This CVE aligns with an *S* attack due to the guessing of TXIDs.

5. **CVE-2020-25684 [92, 93]** identifies an anomaly in dnsmasq, where multiple unanswered queries are forwarded to an upstream server using only 64 random source ports. With multiple TXIDs sharing a port, the effective entropy is reduced to 22 bits, enabling attackers to guess the TXID-port combination through brute force, as seen in *S* attacks. Cisco VPN routers and OpenWRT firmware were among the affected platforms.

6. **CVE-2017-12132 [94]** reveals a vulnerability in the DNS resolver within the glibc library. Exploitation uses large UDP responses, leading to fragmentation, and is categorized as an S_{Frag} attack.

7. **CVE-2008-3217 [95]** describes a flaw in source port randomization in PowerDNS, enabling TXID brute-force attacks. This CVE is thus mapped to an S attack.

8. **CVE-2008-1447 [96]** highlights vulnerabilities in the random generation of source ports and TXIDs in BIND and Microsoft DNS, classifying it as an S attack.

9. **CVE-2008-1454 [97]** documents a vulnerability in Microsoft DNS that allows the resolver to retrieve records outside of the delegated authoritative domain. This vulnerability is exploited via an outside-of-bailiwick rule attack, categorized as B_{OOB}.

10. **CVE-2008-1146 [98]** describes Klein's attack [42] on OpenBSD's BIND, involving guessing 10 TXIDs for a single query, thus making it an S attack.

11. **CVE-2007-2926 [99]** identifies a weak random number generator for TXIDs, which is susceptible to brute-force S attacks.

12. **CVE-2002-2211 [100]** documents a vulnerability in BIND where attackers can send multiple queries for a single resource record, along with spoofed responses. The attack involves sending multiple responses per query, fitting the S attack classification.

13. **CVE-2002-2212 [101]** reports the same vulnerability as CVE-2002-2211, but for Fujitsu UXP/V.

14. **CVE-2002-2213 [102]** reports the same vulnerability as CVE-2002-2211, affecting Infoblox DNS.

Table 2 illustrates the CVEs. An extended mapping of CVEs to POPS's rules can be found in Table 4 Appendix C. Similar to the case of the logic DNS cache poisoning attack, rule Rℓ1 matches the majority of CVEs (78%), followed by Rℓ3 (14%) and Rℓ2 (7%).

CVE	Type	Vendor	POPS
2023-30464 [88]	S	CoreDNS	Rℓ1
2023-28457 [89]	S	Microsoft DNS, Technitium	Rℓ1
2021-43105 [90]	B_{OOB}	Technitium	Rℓ3
2021-3448 [91]	S	dnsmasq	Rℓ1
2020-25684 [92, 93]	S	dnsmasq,Cisco, OpenWRT	Rℓ1
2017-12132 [94]	S_{Frag}	-	Rℓ2
2008-3217 [95]	S	PowerDNS	Rℓ1
2008-1447 [96]	S	BIND, Microsoft DNS	Rℓ1
2008-1454 [97]	B_{OOB}	Microsoft DNS	Rℓ3
2008-1146 [98]	S	OpenBSD's BIND	Rℓ1
2007-2926 [99]	S	ISC BIND	Rℓ1
2002-2211 [100]	S	BIND	Rℓ1
2002-2212 [101]	S	Fujitsu UXP/V	Rℓ1
2002-2213 [102]	S	Infoblox DNS	Rℓ1

Table 2: **CVEs fixed by POPS.** For each CVE, we describe the attack type used to exploit it based on its description, document the vendors affected by the vulnerability, and list the corresponding rule of POPS that matches the CVE.

8 Comparison to Other Tools

In this section we compare POPS to other IDS/IPS tools, Suricata and Snort. Snort and Suricata lack the capability for domain-name aggregation and counting, thus incapable to implement POPS's rules. Additionally, as passive intrusion detection systems (IDS), Snort and Suricata cannot actively block or prevent attacks. In contrast, POPS detection rules and mitigation technique proactively differentiate adversarial packets from benign ones with fine-grained precision.

Suricata [103] is an open-source IDS/IPS and NSM engine that uses multi-threading and signature-based detection to monitor network activity. We compare POPS to Suricata's DNS rules [104, 105] from the OpenWRT ruleset, as the free version available on the Suricata website [103] currently lacks these rules. All following Suricata rules are found to be related to DNS cache poisoning:

1. **SID:2008446** At least 100 DNS responses in 10 seconds. The responses are aggregated only by source IP. The rule also makes sure that there is at least one record in the authority part.

2. **SID:2008447,2008475** Kaminsky attack identification. These rules match sequences of A/NS responses of at least 50 responses in 2 seconds per source IP.

3. **SID:2013894** A tailored rule of attack on Google Brazil's domain (google.com.br). This rule looks for a set of 100 responses in 10 seconds from the same source IP. Each response should contain one record at least in the answer section and in the authority section.

4. **SID:2100253,2100254** Low TTL response. These rules look for responses with no authority section. The first rule specifically looks for PTR response.

Rules 2100253 and 2100254 did not provide sufficient packet details to enable a direct comparison with our system.

These rules were still mentioned as they share some similarities with DNS cache poisoning (i.e., Suricata's phrasing of these rules as "DNS Spoofing", a common synonym to DNS cache poisoning) in Suricata's rule corpus. However, they are (very) weakly related to DNS poisoning and do not mirror any CVE. It is important to note that the aggregation of Suricata's rules is based on IP addresses, while Rℓ1 performs aggregation based on domain names, with a timing threshold of 1 second (compared to Suricata's 2 seconds).

Snort [106, 107] is an open-source IDS/IPS, now maintained by Cisco, that uses a signature-based approach to detect and respond to network threats. To compare with POPS, we identified Snort rules targeting DNS cache poisoning attacks, which can be found by searching "DNS cache" on the Snort website and examining the "snort3-protocol-dns.rules" file from the latest subscribed ruleset. Some of the rules were not detailed, and others were disabled by Snort, so they cannot be compared to POPS. Therefore, we describe four still-running rules with explicit details of DNS cache poisoning attacks:

1. **SID 2008446:** This rule is triggered when the DNS response packet contains more than zero resource records (RRs) in both the answer section and the additional section, with over 100 such packets observed from the same source IP within 10 seconds.
2. **SID 2008447:** This rule indicates a DNS response with 3 RRs. The rule is triggered if there are more than 50 such packets from the same source within 2 seconds.
3. **SID 2008475:** This rule indicates 3 A RRs. It is triggered if more than 50 packets are observed from the same source within 2 seconds.
4. **SID 2008446:** The same as Suricata's SID:2013894 rule.

The aggregation of Snort's is similar to Suricata's, which is less effective than our system as discussed above.

In summary, while both Suricata and Snort provide rules that resemble our Rℓ1 rule, they aggregate packets based on IP addresses. In contrast, our approach aggregates based on domain names, allowing it to capture attacks more efficiently. Furthermore, their rules trigger after a minimum of 2 seconds and 50 packets, whereas our tool operates with a 1-second threshold and just 5 packets, enabling faster detection. Additionally, their rules do not capture fragmentation attacks, which POPS captures. At last, Suricata and Snort are IDS; thus, they only document attacks and do not prevent them. POPS, being an IPS, mitigates the attacks, and moreover the precision of the mitigation is critical to significantly reduce the FP and FN of the whole system.

9 Discussion

Our new system, POPS, assumes that an attacker follows one of three methods—extensive traffic per one query searching for port/TXID, fragmentation, or "out of bailiwick". In the past, a study by Klein et al. [49] presented an attack that assumed the attacker knows the port or TXID from an external source (beyond the DNS protocol itself). Based on this past work, an attacker can adopt a new method: first, the attacker identifies the source port the resolver uses through unspecified techniques. Then, for multiple distinct queries, the attacker sends a single fake response for each, in a packet that adds an authority record and an additional record inside the bailiwick. This way, the attacker evades our rules. By targeting hours with low traffic to the DNS resolver, the attacker can achieve a high ASR. The attacker can also use other templates [108] as well as traditional NS/additional records.

Indeed, this attack does not trigger our rules, and still requires an external technique to know the port or TXID number. However, we conducted an assessment of Tranco's top 1000 domains [81] using simple DNS queries and analyzed the distribution of response types, categorizing them as follows: 46% of the responses included only answer records, 26% included both answer and authority records, 22% had no answer records (but had additional or authority records), and 6% featured answer, authority, and additional records. This preliminary assessment indicates that DNS responses exhibit a typical statistical distribution that can be monitored, and while an attacker could potentially exploit this statistical behavior, the method would require strict adherence to these patterns. A more in-depth analysis would be necessary to establish an additional statistical rule for inclusion in POPS, and this is left for future work.

10 Conclusion

In this study, we introduce POPS, a system designed to protect resolvers against various DNS cache poisoning attacks and mitigate exploits stemming from CVEs. We demonstrate that POPS is more effective for this task compared to Suricata and Snort. Its retrospective capability captures known DNS cache poisoning attacks and provides a foundation for mitigation of future threats, similar to Akamai's recent announcement [24]. For instance, an attack that leverages a combination of fake second fragments and packets scanning source ports (2013B + 2021A) to maximize impact can be effectively detected through the combination of Rℓ1 and Rℓ2. Our simulation of real-world data further validated the system's performance by assessing the rate of false positives generated, thus proving POPS's applicability in practical scenarios.

We acknowledge several limitations of our work. First, while our analysis focuses on DNS cache poisoning attacks, it does not immediately scale to other types of DNS attacks, such as DDoS attacks. Second, our system cannot identify attacks that involve BGP hijacking or MITM techniques combined with DNS cache poisoning. Additionally, attacks that bypass the need to guess both the port and TXID and fall

below the detection threshold will evade our system, however these require advanced tools (malware, middleboxes), outside the threat model. These limitations present opportunities for future research and potential extensions of POPS.

11 Ethics considerations

We evaluated POPS for FP on real data obtained from a published dataset on the Internet [83]. We carefully considered the ethical implications of our analysis and ensured that user privacy was protected. To prevent any harm from using user data in DNS queries, we only retained the responses received by the resolver from DNS authoritative servers, discarding the rest of the data. As a result, no IP addresses or personally identifiable information (PII) from users were used at any stage of the research. All the experiments of POPS were conducted in a controlled laboratory environment, with a dedicated server.

12 Open Science

We fully comply with the USENIX Security 2025 Open Science Policy by openly sharing the research artifacts associated with this work. A comprehensive artifact containing all code, PCAP files, and results is available at [80]. This repository ensures the reproducibility and replicability of our findings. The full repository will be made permanently available after the paper's acceptance, with no licensing restrictions preventing its dissemination. All artifacts will also be submitted to the Artifact Evaluation Committee for review prior to the final submission deadline.

Acknowledgments

We are grateful to the anonymous USENIX Security shepherd and referees for their invaluable, constructive comments and suggestions. We thank Amit Klein and Haya Shulman for helpful discussions. We also thank Micah Sherr (Georgetown University), advisor of the second author during his postdoctoral fellowship. This work was supported in part by an ISF Grant No. 1527/23, a grant from the Blavatnik Interdisciplinary Cyber Research Center (ICRC) at Tel Aviv University, and the Blavatnik Family Fund.

References

[1] P. Mockapetris, "Domain names-concepts and facilities," tech. rep., 1987.

[2] P. V. Mockapetris, "Rfc1035: Domain names-implementation and specification," 1987.

[3] S. Rose and W. Wijngaards, "Dname redirection in the DNS," tech. rep., 2012.

[4] M. Andrews, "Negative caching of DNS queries (DNS NCACHE)," tech. rep., 1998.

[5] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "Protocol modifications for the DNS security extensions," tech. rep., 2005.

[6] S. Bortzmeyer, "DNS query name minimisation to improve privacy," tech. rep., 2016.

[7] S. Y. Chau, O. Chowdhury, V. Gonsalves, H. Ge, W. Yang, S. Fahmy, and N. Li, "Adaptive deterrence of DNS cache poisoning," in *Security and Privacy in Communication Networks: 14th International Conference, SecureComm 2018, Singapore, Singapore, August 8-10, 2018, Proceedings, Part II*, pp. 171–191, Springer, 2018.

[8] ROAR Media, "What the .lk domain registry hack means for the future of cybersecurity in sri lanka," 2021.

[9] Securelist, "Massive DNS poisoning attacks in brazil," 2011.

[10] S. Cherry, "Panix attack," 2005.

[11] D. Kaminsky, "Dns critical infrastructure." Presentation at Black Hat DC 2009, 2009.

[12] C. Schuba and E. H. Spafford, "Addressing weaknesses in the domain name system protocol," *Master's thesis, Purdue University*, 1993.

[13] D. Kaminsky, "Black ops 2008: It's the end of the cache as we know it," *Black Hat USA*, vol. 2, 2008.

[14] R. Morillo, J. Furuness, C. Morris, J. Breslin, A. Herzberg, and B. Wang, "Rov++: Improved deployable defense against bgp hijacking.," in *NDSS*, 2021.

[15] M. Conti, N. Dragoni, and V. Lesyk, "A survey of man in the middle attacks," *IEEE communications surveys & tutorials*, vol. 18, no. 3, pp. 2027–2051, 2016.

[16] H. Berger, A. Z. Dvir, and M. Geva, "A wrinkle in time: a case study in DNS poisoning," *International Journal of Information Security*, vol. 20, pp. 313–329, 2021.

[17] X. Li, C. Lu, B. Liu, Q. Zhang, Z. Li, H. Duan, and Q. Li, "The maginot line: Attacking the boundary of {DNS} caching protection," in *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 3153–3170, 2023.

[18] A. Herzberg and H. Shulman, "Security of patched DNS," in *Computer Security–ESORICS 2012: 17th European Symposium on Research in Computer Security, Pisa, Italy, September 10-12, 2012. Proceedings 17*, pp. 271–288, Springer, 2012.

[19] G. Pazi, D. Touitou, A. Golan, and Y. Afek, "Protecting against spoofed DNS messages," June 14 2005. US Patent 6,907,525.

[20] Y. Afek, A. Bremler-Barr, and L. Shafir, "Network anti-spoofing with SDN data plane," in *IEEE INFOCOM 2017-IEEE conference on computer communications*, pp. 1–9, IEEE, 2017.

[21] P. Vixie and V. Schryver, "DNS response rate limiting (DNS RRL), 2012."

[22] X. Li, W. Xu, B. Liu, M. Zhang, Z. Li, J. Zhang, D. Chang, X. Zheng, C. Wang, J. Chen, *et al.*, "Tudoor attack: Systematically exploring and exploiting logic vulnerabilities in DNS response pre-processing with malformed packets," in *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 46–46, IEEE Computer Society, 2023.

[23] P. Jeitner and H. Shulman, "Injection attacks reloaded: Tunnelling malicious payloads over {DNS}," in *30th USENIX Security Symposium (USENIX Security 21)*, pp. 3165–3182, 2021.

[24] J. S. Bruce Van Nice, Mark Dokter, "Keytrap highlights the need for enduring DNS defenses for service providers," 2024. Accessed: 2024-08-30.

[25] Y. Afek, A. Bremler-Barr, S. Danino, and Y. Shavitt, "A flushing attack on the {DNS} cache," in *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 2299–2314, 2024.

[26] P. Flajolet, D. Gardy, and L. Thimonier, "Birthday paradox, coupon collectors, caching algorithms and self-organizing search," *Discrete Applied Mathematics*, vol. 39, no. 3, pp. 207–229, 1992.

[27] "Frag-dns." https://indico.dns-oarc.net/event/42/contributions/909/attachments/866/1568/Frag-DNS-OARC-2022.pdf, 2022. Presentation at DNS-OARC 2022, accessed: 2025-01-19.

[28] D. Dagon, M. Antonakakis, P. Vixie, T. Jinmei, and W. Lee, "Increased DNS forgery resistance through 0x20-bit encoding: security via leet queries," in *Proceedings of the 15th ACM conference on Computer and communications security*, pp. 211–222, 2008.

[29] Sophos News, "Serious security: How deliberate typos might improve dns security," January 2023.

[30] T. Dai, P. Jeitner, H. Shulman, and M. Waidner, "From ip to transport and beyond: cross-layer attacks against applications," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pp. 836–849, 2021.

[31] H. Ballani, P. Francis, and X. Zhang, "A study of prefix hijacking and interception in the internet," *ACM SIGCOMM Computer Communication Review*, vol. 37, no. 4, pp. 265–276, 2007.

[32] H. Birge-Lee, Y. Sun, A. Edmundson, J. Rexford, and P. Mittal, "Bamboozling certificate authorities with {BGP}," in *27th USENIX Security Symposium (USENIX Security 18)*, pp. 833–849, 2018.

[33] F. Alharbi, J. Chang, Y. Zhou, F. Qian, Z. Qian, and N. Abu-Ghazaleh, "Collaborative client-side DNS cache poisoning attack," in *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*, pp. 1153–1161, IEEE, 2019.

[34] T. Ma, C. Xu, Z. Zhou, X. Kuang, L. Zhong, and L. A. Grieco, "Intelligent-driven adapting defense against the client-side DNS cache poisoning in the cloud," in *GLOBECOM 2020-2020 IEEE Global Communications Conference*, pp. 1–6, IEEE, 2020.

[35] MITRE, "Cwe-89: Improper neutralization of special elements used in an sql command ('sql injection')," 2006.

[36] K. Elshazly, Y. Fouad, M. Saleh, and A. Sewisy, "A survey of sql injection attack detection and prevention," *Journal of Computer and Communications*, vol. 2014, 2014.

[37] Z. Qian, Z. M. Mao, and Y. Xie, "Collaborative tcp sequence number inference attack: how to crack sequence number under a second," in *Proceedings of the 2012 ACM conference on Computer and communications security*, pp. 593–604, 2012.

[38] Z. Qian and Z. M. Mao, "Off-path tcp sequence number inference attack-how firewall middleboxes reduce security," in *2012 IEEE Symposium on Security and Privacy*, pp. 347–361, IEEE, 2012.

[39] W. Chen and Z. Qian, "{Off-Path}{TCP} exploit: How wireless routers can jeopardize your secrets," in *27th USENIX Security Symposium (USENIX Security 18)*, pp. 1581–1598, 2018.

[40] Y. Cao, Z. Qian, Z. Wang, T. Dao, S. V. Krishnamurthy, and L. M. Marvel, "{Off-Path}{TCP} exploits: Global rate limit considered dangerous," in *25th USENIX Security Symposium (USENIX Security 16)*, pp. 209–225, 2016.

[41] V. Sacramento, "Vulnerability in the sending requests control of bind versions 4 and 8 allows DNS spoofing," 2002.

[42] A. Klein, "Bind 9 DNS cache poisoning," *Report, Trusteer, Ltd*, vol. 3, 2007.

[43] A. Herzberg and H. Shulman, "Fragmentation considered poisonous, or: One-domain-to-rule-them-all. org," in *2013 IEEE Conference on Communications and Network Security (CNS)*, pp. 224–232, IEEE, 2013.

[44] A. Herzberg and H. Shulman, "Vulnerable delegation of DNS resolution," in *Computer Security–ESORICS 2013: 18th European Symposium on Research in Computer Security, Egham, UK, September 9-13, 2013. Proceedings 18*, pp. 219–236, Springer, 2013.

[45] A. Herzberg and H. Shulman, "Socket overloading for fun and cache-poisoning," in *Proceedings of the 29th Annual Computer Security Applications Conference*, pp. 189–198, 2013.

[46] X. Zheng, C. Lu, J. Peng, Q. Yang, D. Zhou, B. Liu, K. Man, S. Hao, H. Duan, and Z. Qian, "Poison over troubled forwarders: A cache poisoning attack targeting {DNS} forwarding devices," in *29th USENIX Security Symposium (USENIX Security 20)*, pp. 577–593, 2020.

[47] K. Man, Z. Qian, Z. Wang, X. Zheng, Y. Huang, and H. Duan, "DNS cache poisoning attack reloaded: Revolutions with side channels," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1337–1350, 2020.

[48] T. Dai, H. Shulman, and M. Waidner, "DNS-over-TCP considered vulnerable," in *Proceedings of the Applied Networking Research Workshop*, pp. 76–81, 2021.

[49] A. Klein, "Cross layer attacks and how to use them (for DNS cache poisoning, device tracking and more)," in *2021 IEEE Symposium on Security and Privacy (SP)*, pp. 1179–1196, IEEE, 2021.

[50] P. Jeitner, H. Shulman, L. Teichmann, and M. Waidner, "{XDRI} attacks-and-how to enhance resilience of residential routers," in *31st USENIX Security Symposium (USENIX Security 22)*, pp. 4473–4490, 2022.

[51] E. Heftrig, H. Shulman, and M. Waidner, "Poster: Off-path DNSSEC downgrade attacks," in *Proceedings of the ACM SIGCOMM 2023 Conference*, pp. 1120–1122, 2023.

[52] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS security introduction and requirements," tech. rep., 2005.

[53] R. Barnes, "Use cases and requirements for DNS-based authentication of named entities (dane)," tech. rep., 2011.

[54] O. Gudmundsson, "Adding acronyms to simplify conversations about DNS-based authentication of named entities (dane)," tech. rep., 2014.

[55] L. Zhu, D. Wessels, A. Mankin, and J. Heidemann, "Measuring dane tlsa deployment," in *Traffic Monitoring and Analysis: 7th International Workshop, TMA 2015, Barcelona, Spain, April 21-24, 2015. Proceedings 7*, pp. 219–232, Springer, 2015.

[56] M. Dempsky, "DNScurve: Link-level security for the domain name system," *Work in Progress, draft-dempsky-dnscurve-01*, 2010.

[57] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. Hoffman, "Specification for DNS over transport layer security (tls)," tech. rep., 2016.

[58] T. Reddy, D. Wing, and P. Patil, "DNS over datagram transport layer security (dtls)," tech. rep., 2017.

[59] P. Hoffman and P. McManus, "DNS queries over HTTPS (DoH)," tech. rep., 2018.

[60] E. Blidborg, *Cache Poisoning in DNS over HTTPS clients*. PhD thesis, KTH, Stockholm, Sweden, 2020.

[61] N. Alexiou, S. Basagiannis, P. Katsaros, T. Dashpande, and S. A. Smolka, "Formal analysis of the kaminsky DNS cache-poisoning attack using probabilistic model checking," in *2010 IEEE 12th International Symposium on High Assurance Systems Engineering*, pp. 94–103, IEEE, 2010.

[62] M. Kwiatkowska, G. Norman, and D. Parker, "Prism: Probabilistic symbolic model checker," in *International Conference on Modelling Techniques and Tools for Computer Performance Evaluation*, pp. 200–204, Springer, 2002.

[63] T. S. Deshpande, *Formal Analysis of DNS Attacks and Their Countermeasures Using Probabilistic Model Checking*. PhD thesis, State University of New York at Stony Brook, 2013.

[64] G. J. Holzmann, "The model checker spin," *IEEE Transactions on software engineering*, vol. 23, no. 5, pp. 279–295, 1997.

[65] W. Zhanga, M. Yanga, X. Zhanga, and H. Shia, "Model checking the DNS under DNS cache-poisoning attacks using spin," *SCIENCEASIA*, vol. 42, pp. 49–55, 2016.

[66] A. Laraba, J. François, I. Chrisment, S. R. Chowdhury, and R. Boutaba, "Detecting multi-step attacks: A modular approach for programmable data plane," in *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–9, IEEE, 2022.

[67] J. L. Peterson, "Petri nets," *ACM Computing Surveys (CSUR)*, vol. 9, no. 3, pp. 223–252, 1977.

[68] M. Antonakakis, D. Dagon, X. Luo, R. Perdisci, W. Lee, and J. Bellmor, "A centralized monitoring infrastructure for improving DNS security," in *Recent Advances in Intrusion Detection: 13th International Symposium, RAID 2010, Ottawa, Ontario, Canada, September 15-17, 2010. Proceedings 13*, pp. 18–37, Springer, 2010.

[69] Y. Jin, M. Tomoishi, and S. Matsuura, "A detection method against DNS cache poisoning attacks using machine learning techniques: Work in progress," in *2019 IEEE 18th International Symposium on Network Computing and Applications (NCA)*, pp. 1–3, IEEE, 2019.

[70] S. L. Feibish, Y. Afek, A. Bremler-Barr, E. Cohen, and M. Shagam, "Mitigating DNS random subdomain DDoS attacks by distinct heavy hitters sketches," *HotWeb*, pp. 8:1 – 8:6, 2017. arXiv preprint arXiv:1612.02636.

[71] Y. Ozery, A. Nadler, and A. Shabtai, "Information-based heavy hitters for real-time DNS data exfiltration detection," 2024.

[72] G. Duan, Q. Li, Z. Zhang, D. Zhao, G. Xie, Y. Yang, Z. Yuan, Y. Jiang, and M. Xu, "DNSGuard: In-network defense against DNS attacks," *IEEE Transactions on Dependable and Secure Computing*, 2024.

[73] G. Cormode, "Count-min sketch.," 2009.

[74] P. Mockapetris, "Domain Names - Implementation and Specification." Request for Comments (RFC), November 1987. Obsoleted by RFC 2181, RFC 4033, RFC 4034, RFC 4035, RFC 3597, RFC 1123.

[75] P. Vixie, O. Gudmundsson, M. Majkowski, and E. Lewis, "Extension Mechanisms for DNS (EDNS(0))." RFC 6891, 2013.

[76] APNIC, "Revisiting DNS and UDP truncation." https://blog.apnic.net/2024/07/15/revisiting-dns-and-udp-truncation/, July 2024. Accessed: 2024-08-27.

[77] G. C. Moura, M. Müller, M. Davids, M. Wullink, and C. Hesselman, "Fragmentation, truncation, and timeouts: are large dns messages falling to bits?," in *Passive and Active Measurement: 22nd International Conference, PAM 2021, Virtual Event, March 29–April 1, 2021, Proceedings 22*, pp. 460–477, Springer, 2021.

[78] P. Bhowmick, M. I. Ashiq, C. Deccio, and T. Chung, "Ttl violation of DNS resolvers in the wild," in *International Conference on Passive and Active Network Measurement*, pp. 550–563, Springer, 2023.

[79] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "Openflow: enabling innovation in campus networks," *ACM SIGCOMM computer communication review*, vol. 38, no. 2, pp. 69–74, 2008.

[80] "Poisoning prevention system." https://zenodo.org/records/15606846.

[81] V. L. Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczyński, and W. Joosen, "Tranco: A research-oriented top sites ranking hardened against manipulation," *arXiv preprint arXiv:1806.01156*, 2018.

[82] X. Li, D. Wu, H. Duan, and Q. Li, "DNSBOMB: A new practical-and-powerful pulsing dos attack exploiting DNS queries-and-responses," in *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 253–253, IEEE Computer Society, 2024.

[83] M. Singh, M. Singh, and S. Kaur, "10 Days DNS Network Traffic from April-May, 2016," 2019.

[84] "National Institute of Standards and Technology." https://www.nist.gov/. Accessed: September 29, 2024.

[85] "Vulnerability search results for DNS cache poisoning." https://nvd.nist.gov/vuln/search/results?isCpeNameSearch=false&query=dns+cache+poisoning&results_type=overview&form_type=Basic&search_type=all&startIndex=0, 2024. Accessed: 2024-09-29.

[86] "Vulnerability search results for DNS spoofing." https://nvd.nist.gov/vuln/search/results?isCpeNameSearch=false&query=dns+spoofing&results_type=overview&form_type=Basic&search_type=all&startIndex=0, 2024. Accessed: 2024-09-29.

[87] National Vulnerability Database, "CVE-2008-3280," 2008. Accessed: 2024-09-29.

[88] "CVE-2023-30464: Apache airflow XSS vulnerability." https://nvd.nist.gov/vuln/detail/CVE-2023-30464, 2023.

[89] "CVE-2023-28457: Docker CLI vulnerability." https://nvd.nist.gov/vuln/detail/CVE-2023-28457, 2023.

[90] "CVE-2021-43105." https://nvd.nist.gov/vuln/detail/CVE-2021-43105, 2021.

[91] "CVE-2021-3448." https://nvd.nist.gov/vuln/detail/CVE-2021-3448, 2021.

[92] "CVE-2020-25684." https://nvd.nist.gov/vuln/detail/CVE-2020-25684, 2020.

[93] J. tech, "DNSpooq," 2021.

[94] "CVE-2017-12132." https://nvd.nist.gov/vuln/detail/CVE-2017-12132, 2017.

[95] "CVE-2008-3217." https://nvd.nist.gov/vuln/detail/CVE-2008-3217, 2008.

[96] "CVE-2008-1447." https://nvd.nist.gov/vuln/detail/cve-2008-1447, 2008.

[97] "CVE-2008-1454." https://nvd.nist.gov/vuln/detail/cve-2008-1454, 2008.

[98] "CVE-2008-1146." https://nvd.nist.gov/vuln/detail/cve-2008-1146, 2008.

[99] "CVE-2007-2926." https://nvd.nist.gov/vuln/detail/cve-2007-2926, 2007.

[100] "CVE-2002-2211." https://nvd.nist.gov/vuln/detail/cve-2002-2211, 2002.

[101] "CVE-2002-2212." https://nvd.nist.gov/vuln/detail/cve-2002-2212, 2002.

[102] "CVE-2002-2213." https://nvd.nist.gov/vuln/detail/cve-2002-2213, 2002.

[103] OISF, "Suricata," 2024.

[104] Suricata, "Suricata DNS events," 2018.

[105] Suricata, "Suricata emerging DNS events," 2017.

[106] J. Koziol, *Intrusion detection with Snort*. Sams Publishing, 2003.

[107] B. Caswell, J. Beale, and A. Baker, *Snort intrusion detection and prevention toolkit*. Syngress, 2007.

[108] A. Klein, H. Shulman, and M. Waidner, "Internet-wide study of DNS cache injections," in *IEEE INFOCOM 2017-IEEE Conference on Computer Communications*, pp. 1–9, IEEE, 2017.

[109] S. L. Feibish, Y. Afek, A. Bremler-Barr, E. Cohen, and M. Shagam, "Mitigating dns random subdomain ddos attacks by distinct heavy hitters sketches," in *Proceedings of the fifth ACM/IEEE workshop on hot topics in web systems and technologies*, pp. 1–6, 2017.

A Appendix - Full Analysis - Rℓ1

In this appendix, we provide the full analysis of CMS, dwsHH, and WS algorithms. First, we describe the algorithms we compared to meet Rℓ1. Then, we describe the comparative analysis of the algorithms. The analysis is based on Table 3, for memory usage, inference time, and error rates of the three algorithms.

Method	Memory	Error	Inference
CMS	$w \times d \times$ |counter|	$\varepsilon = \frac{e}{w}$	$O(d)$
dwsHH	$O(k\ell \log m)$	$O\left(\frac{1}{k}\sum_y w_y + \frac{w_y}{\sqrt{2\ell}}\right)$	$O(\log N)$
WS	$O(\tau \sum_y w_y \cdot \ell \log m)$	$O\left(\tau^{-1} + \frac{w_y}{\sqrt{2\ell}}\right)$	$O(\tau N)$

Table 3: Comparison of memory usage (Memory), error rates (Error), and inference time (Inference) for CMS, dwsHH, and WS. Specific details are presented in Sections A.2-A.4.

A.1 Algorithms

Count-Min Sketch (CMS) is a compact data structure used to estimate item frequencies in a data stream. It consists of a table with multiple rows, each linked to a unique hash function, and multiple columns representing counters. When an item arrives, it is hashed using each function, and the resulting hash values indicate which counters to increment. The item's frequency is estimated by taking the minimum value among these counters. CMS's memory usage depends on the desired error rate (ε) and confidence level ($1-\delta$), which determine the number of hash functions (depth, d) and counters per hash function (width, w).

Fixed-size Distinct Weighted Sampling for Heavy Hitters (dwsHH) identifies frequently occurring items (heavy hitters) in a data stream using a fixed memory size. It maintains a weighted sample of items based on their frequency. When a new item arrives, the algorithm decides whether to add it to the sample, depending on its weight and the sample's current state. The memory usage is influenced by the cache size (k), the sampling window length (ℓ), and the logarithm of the monitored item count (m). The sample dynamically updates, keeping the most significant items while minimizing memory use.

Fixed-threshold Weighted Sampling (WS) identifies significant or "heavy" items in a data stream using a fixed threshold (τ). Items are sampled if their weight or frequency exceeds this threshold, which helps focus on important items without tracking every item in the stream. Upon the arrival of a new item, its weight is compared to τ; if it exceeds the threshold, it is added to the sample. This method optimizes memory and computation by focusing on items that significantly affect data distribution, reducing errors in frequency estimation for

less significant items. The following calculations rely on the Count-Min Sketch framework as described in [73], and heavy hitters in [109].

A.2 Memory Usage Calculation

1. **CMS**: The w and d values are fixed, and for optimum utility (as described in Section 5), we picked $d = 5, w = 200$. The counter is given as an int (4 bits). The resulting memory usage is 4k bits.

2. **Fixed-size dwsHH**: The memory usage is calculated as $O(k\ell \log m)$. Assuming $k = 100$ and $\ell = 32$, and $m \approx 0.1N$:

   For 10 domains: $100 \times 32 \times \log(1) = 3.2K$ bits

   For 100 domains: $100 \times 32 \times \log(10) = 3.2K$ bits

   For 1K domains: $100 \times 32 \times \log(100) = 6.4K$ bits

   For 10K domains: $100 \times 32 \times \log(1000) = 9.6K$ bits

3. **Fixed-threshold WS**: The memory usage is calculated as $O(\tau \sum_y w_y \cdot \ell \log m)$. With $\tau = 0.01$:

   For 10 domains: $0.01 \times 10 \times 32 \times \log(1) = 3.2$ bytes

   For 100 domains: $0.01 \times 100 \times 32 \times \log(10) = 32$ bytes

   For 1K domains: $0.01 \times 1K \times 32 \times \log(100) = 640$ bytes

   For 10K domains: $0.01 \times 10K \times 32 \times \log(10) = 9.6K$ bytes

The comparison of memory usage is presented in Fig. 5. It is shown that the memory usage of CMS is high but constant, and the heavy hitters' memory usage is increasing. With a high number of domains (10k), the CMS utilizes less memory.

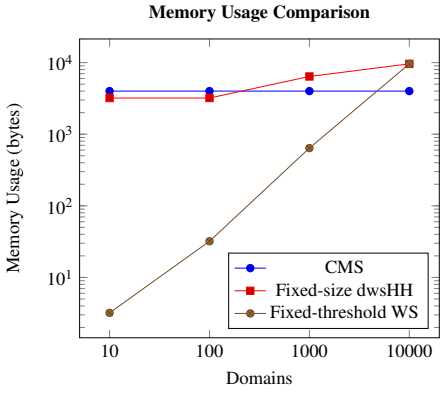

Figure 5: Memory usage comparison, between CMS and Heavy Hitters. The CMS algorithm gets high memory usage but is constant. At its peak, the CMS is less memory-consumable than the heavy hitters.

A.3 Error Rates Calculation

All parameters for the three algorithms are the same as in the previous subsection.

1. **Count-Min Sketch (CMS)**:

$$\varepsilon = \frac{e}{w} \approx \frac{2.718}{200} \approx 0.01$$

2. **Fixed-size dwsHH**: The error formula is given by:

$$\text{Error} = \frac{1}{k}\sum_y w_y + \frac{w_y}{\sqrt{2\ell}}$$

   Assuming $w_y = 1$ for all y and $\ell = k$, we calculate the error for different cache sizes k:
   For 10 domains: $\frac{1}{10}\sum_y 1 + \frac{1}{\sqrt{2\times 10}} = \frac{10}{10} + \frac{1}{\sqrt{20}} = 1 + \frac{1}{4.47} \approx 1 + 0.223 = 1.223$
   For 100 domain: $\frac{1}{100}\sum_y 1 + \frac{1}{\sqrt{2\times 100}} = \frac{100}{100} + \frac{1}{\sqrt{200}} = 1 + \frac{1}{14.14} \approx 1 + 0.071 = 1.071$
   For 1000 domains: $\frac{1}{1000}\sum_y 1 + \frac{1}{\sqrt{2\times 1000}} = \frac{1000}{1000} + \frac{1}{\sqrt{2000}} = 1 + \frac{1}{44.72} \approx 1 + 0.022 = 1.022$
   For 10000 domains: $\frac{1}{10000}\sum_y 1 + \frac{1}{\sqrt{2\times 10000}} = \frac{10000}{10000} + \frac{1}{\sqrt{20000}} = 1 + \frac{1}{141.42} \approx 1 + 0.007 = 1.007$

3. **Fixed-threshold WS**: The error bound is calculated as $\tau^{-1} + \frac{w_y}{\sqrt{2\ell}} = 0.01^{-1} + \frac{1}{\sqrt{64}} = 100.125$:

The comparison of error rates is presented in Fig. 6. It can be seen that the error rate of CMS ($\sim$0.01) is less than the error rates of the heavy hitters (more than 1 and more than 100, respectively).

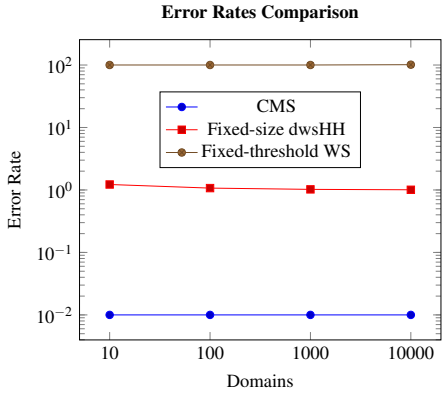

Figure 6: Error rates comparison, between CMS and Heavy Hitters. The error rate of CMS is well under the heavy hitters, in order of magnitude (~ 0.01 vs more than 1, or more than 100).

A.4 Inference Time Calculation

All parameters remain consistent with those in the previous subsections.

1. **Count-Min Sketch (CMS)**: The query time is $O(d)$, with $d = 5$, involving 5 hash lookups.
2. **Fixed-size dwsHH**: The query time is $O(\log n)$ for keys not in cache, and 2 accesses for keys in the cache.
3. **Fixed-threshold WS**: The query time is $O(\tau N)$:

$$\text{For 10 domains: } 0.01 \times 10 = 0.1$$

$$\text{For 100 domains: } 0.01 \times 100 = 1$$

$$\text{For 1000 domains: } 0.01 \times 1000 = 10$$

$$\text{For 10K domains: } 0.01 \times 10000 = 100$$

As shown in Fig. 7, the comparison of inference times indicates that the CMS query time stays constant—requiring only five hash function calculations—whereas the query time for the heavy hitters' methods increases with the number of domains.

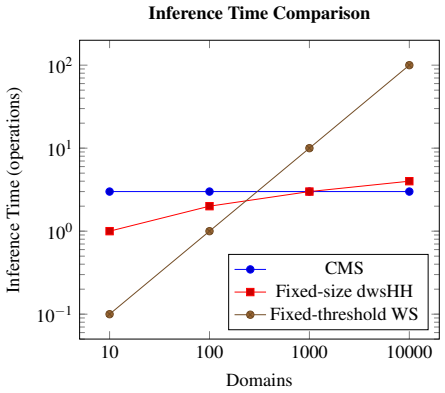

Figure 7: Inference time comparison, between CMS and Heavy Hitters. CMS's inference time is constant at 5, where the other algorithms increase based on the number of domains.

B Appendix - Full Algorithm

C CVE-to-POPS- Extended

In this appendix, we provide additional explanation of the mapping of each CVE to its POPS rule. The details are disclosed in Table 4.

Algorithm 1 POPS

Require: Set of packets $P = \{p_1, p_2, \ldots, p_n\}$, Count-Min Sketch (CMS), Threshold τ, Window size W, initial time t, domain map $domainMap$, global $action$
Ensure: Global constants: CMS, τ

for each packet $p_i \in P$ **do** ▷ Iterate through all packets
  if $R\ell1(p_i, domainMap)$ **or** $R\ell2(p_i)$ **or** $R\ell3(p_i)$ **then**
    if $p_i != NULL$ **then**
      $action \leftarrow$ "$truncate$"
      p_i.TC $\leftarrow 1$ ▷ Set Truncated bit
      p_i.Ans $\leftarrow$ blank ▷ Clear answer record
      p_i.Auth $\leftarrow$ blank ▷ Clear auth record
      p_i.Add $\leftarrow$ blank ▷ Clear add record
    end if
  else
    $action \leftarrow$ "$forward$" ▷ No rule fired
  end if
  Send p_i to p_i.dest ▷ Send packet to its destination
end for

Algorithm 2 Rℓ1 Function - Probabilistic Count of Domains

function R$\ell1(p, domainMap)$ ▷ Initialize result as False
  CMS.Add(p.domain) ▷ Update CMS with p
  if CMS.Freq(p) $> \tau$ **then**
    if $domainMap[p.domain]$ == true **then**
      return true
    else
      $domainMap[p.domain] \leftarrow true$ ▷ Signal the domain is from now suspicious
      return true ▷ Now, truncate the packet
    end if
  end if
  if $p.Time - t \mod W > 0$ **then** ▷ window t/o
    $CMS \leftarrow New(CMS)$ ▷ init the CMS table
    $domainMap \leftarrow domainMap$ ▷ init map
    $t \leftarrow p.Time$ ▷ Init the time variable
  end if
  return false
end function

Algorithm 3 Rℓ2 Function - Check for Fragmentation

function R$\ell2(p)$ ▷ Initialize result as False
  if p.offset == 0 & p.MF == **true then** ▷ 1st frag
    return true
  else$p$.offset > 0 ▷ 2nd frags
    $p \leftarrow NULL$
  end if
  return false
end function

Algorithm 4 Rℓ3 Function - Bailiwick Rule Compliance

```
1: function Rℓ3(p)
2:     is_valid ← true
3:     for all record ∈ p.answers do
4:         if not IsWithinBailiwick(record.name, p.queryname) then
5:             is_valid ← false
6:             break
7:         end if
8:     end for
9:     if is_valid then
10:        for all record ∈ p.authoritative do
11:            if not IsWithinBailiwick(p.queryname, record.name) then
12:                is_valid ← false
13:                break
14:            end if
15:        end for
16:    end if
17:    if is_valid then
18:        for all record ∈ p.additional do
19:            if not IsWithinBailiwick(record.name, p.queryname) then
20:                is_valid ← false
21:                break
22:            end if
23:        end for
24:    end if
25:    if ¬is_valid then return true
26:    else return false
27:    end if
28: end function
29: function IsWithinBailiwick(domain, origin)
30:    is_within ← false
31:    if EndsWith(domain, origin) then
32:        is_within ← true
33:    end if
34: return is_within
35: end function
```

CVE	POPS	Explanation
2023-30464 [88], 2023-28457 [89]	Rℓ1	POPS identifies same domain, different TXID/ports pkts.
2021-43105 [90]	Rℓ3	POPS identifies an out-of-bailiwick record, where auth is not in the same domain as the query/response.
2021-3448 [91]	Rℓ1	POPS identifies same domain, different TXIDs pkts (fixed port).
2020-25684 [92, 93]	Rℓ1	POPS identifies same domain, different TXIDs pkts. Port not validated. Attacker fakes TXIDs.
2017-12132 [94]	Rℓ2	When EDNS is enabled in the glibc resolver, it can elicit large responses that cause fragmentation. POPS identifies fragmentation attack.
2008-3217 [95]	Rℓ1	Low randomness in port selection leads to predictable src port. POPS identifies subsequent same domain, different TXIDs ptks.
2008-1447 [96]	Rℓ1	Insufficient randomness of DNS TXIDs and port, leading to same domain, different TXID/port's attack (which POPS identifies).
2008-1454 [97]	Rℓ3	Same as 2021-43105.
2008-1146 [98]	Rℓ1	Low randomness in TXID selection leads to predictable TXID. POPS identifies subsequent same domain, different ports pkts.
2007-2926 [99]	Rℓ1	Same as 2008-1146, different vendor.
2002-2211 [100]	Rℓ1	Birthday attack - many queries & fake replies for the same domain. POPS identifies same domain, different TXID/port pkts.
2002-2212 [101], 2002-2213 [102]	Rℓ1	Same as 2002-2211, other vendors.

Table 4: **CVEs covered by POPS.** For each CVE, we identify the corresponding POPS rule that matches it, and explain both the nature of the attack and how POPS detects it.